

Mathematical Background for `AccumulatR`

Niek Stevenson

Abstract

`AccumulatR` is built around one simple idea: each latent process has a random finishing time, and responses are defined by rules about the relative order of those finishing times. This note gives a compact mathematical description of that framework. The aim is not maximal generality. The aim is a notation that is simple enough to read from start to finish and precise enough to match the package.

1 Overview

The framework has four modeling layers.

1. A *leaf* is a single latent finishing-time process.
2. A *pool* combines several sources into one new source.
3. An *outcome rule* says when a modeled response becomes available.
4. An *observation layer* turns latent outcomes into the data we actually record, for example through mixtures, remapped labels, missing response times, or ranked responses.

The evaluator has one additional layer. It lowers the model to symbolic transition scenarios, turns target and competitor relations into one order region, simplifies that region, and then compiles it to numerical expressions. Simple models reduce to familiar products of densities, distribution functions, and survival functions. More structured models are still handled by the same order-region machinery rather than by separate special-case formulas.

2 Leaves and Basic Notation

A leaf is the basic latent process in the model. If leaf a finishes at random time T_a , write

$$F_a(t) = \Pr(T_a \leq t), \quad f_a(t) = \frac{d}{dt}F_a(t), \quad S_a(t) = 1 - F_a(t).$$

So $F_a(t)$ is the probability that leaf a has finished by time t , $f_a(t)$ is the density that it finishes at time t , and $S_a(t)$ is the probability that it has not finished by time t .

The same notation can be used for any source whose finishing-time distribution has already been defined. In this note, a *source* means a leaf or pool. Outcome rules are slightly more general: they may become true in several different ways, and those ways may share latent sources with other rules.

Different leaves may have different finishing-time distributions. The package currently supplies the families

lognormal, gamma, exgauss, LBA, RDM.

The exact calculations below do not depend on a particular family. They only require that each leaf supplies the needed density and distribution functions.

3 Pools

A pool turns several sources into one new source. Suppose pool P has members with finishing times $T_1, \dots, T_n$, and suppose the pool finishes when the k th member finishes. Then T_P is the k th order statistic of $T_1, \dots, T_n$.

The defining identity is

$$F_P(t) = \Pr(T_P \leq t) = \Pr(\text{at least } k \text{ of } T_1, \dots, T_n \text{ are } \leq t). \quad (1)$$

That is the cleanest way to think about a pool: by time t , the pool has finished exactly when enough of its members have finished.

If the pool members are exchangeable, with common CDF F and density f , then

$$F_P(t) = \sum_{m=k}^n \binom{n}{m} F(t)^m S(t)^{n-m}, \quad (2)$$

and

$$f_P(t) = \frac{n!}{(k-1)!(n-k)!} f(t) F(t)^{k-1} S(t)^{n-k}. \quad (3)$$

These are the usual formulas for the k th finisher.

The package also allows pools with non-identical members and nested pools. In those cases the product formulas are no longer the main definition. The pool is compiled as order constraints over its members. If a pool shares a member with another source or rule, that shared latent time remains shared in the compiled region.

4 Outcome Rules

An outcome rule says when a modeled response becomes available. If E is an outcome rule, then it is useful to think of E as a union of transition scenarios. A transition scenario records

- which source releases the rule at the transition time,
- which other requirements must already be ready,
- which guards or absence conditions must still hold,
- and which order facts are implied by that way of becoming true.

For simple independent rules, these scenarios collapse to familiar formulas. For shared or nested rules, the scenarios are kept as symbolic order regions.

4.1 Direct Outcomes

The simplest outcome rule is a direct readout from one source. In that case, the outcome simply inherits the finishing-time distribution of that source. Nothing new is added mathematically.

4.2 `first_of()`

If

$$E = \text{first_of}(E_1, \dots, E_m),$$

then the rule becomes true when the first child becomes true. If the children are independent and do not share latent sources, then

$$F_E(t) = 1 - \prod_{r=1}^m S_{E_r}(t), \quad f_E(t) = \sum_{r=1}^m f_{E_r}(t) \prod_{q \neq r} S_{E_q}(t). \quad (4)$$

The density has the usual interpretation: one child finishes at time t , and all the others are still unfinished at time t .

In the general case, the compiler does not assume this product form. It creates one transition scenario for each child being the first available route, then adds order constraints saying that the other children are not earlier.

4.3 `all_of()`

If

$$E = \text{all_of}(E_1, \dots, E_m),$$

then the rule becomes true when the last required child becomes true. If the children are independent and do not share latent sources, then

$$F_E(t) = \prod_{r=1}^m F_{E_r}(t), \quad f_E(t) = \sum_{r=1}^m f_{E_r}(t) \prod_{q \neq r} F_{E_q}(t). \quad (5)$$

Here one child finishes at time t , and all the others have already finished by time t .

In the general case, the compiler again works by scenarios. Each scenario says which child supplies the final release time and which children must already be ready. If two rules share that final release source, the equality is kept as a model tie rather than being discarded as a zero-probability boundary.

4.4 `none_of()`

`none_of()` is an absence condition. If a rule requires `none_of(B)` at time t , then B must not have become true before that time. In the simplest case, where B is an independent source, this is just

$$\Pr(B \text{ has not finished by time } t) = S_B(t).$$

For example, if

$$E = \text{all_of}(A, \text{none_of}(B)),$$

and A and B are independent direct sources, then the transition density is

$$f_E(t) = f_A(t)S_B(t), \quad F_E(t) = \int_0^t f_A(u)S_B(u) du.$$

For composite or shared blockers, `none_of()` is represented as an order constraint. It does not introduce a new observed race winner, but it can still share latent state with the target rule or with competitors.

4.5 `inhibit()`

The blocking rule is written as

$$G = \text{inhibit}(A, B).$$

Its meaning is: count A only if blocker B is not before A . If A and B are independent direct sources, then

$$f_G(t) = f_A(t)S_B(t), \quad F_G(t) = \int_0^t f_A(u)S_B(u) du. \quad (6)$$

So A finishes at time t , while B is still unfinished at time t .

In the general case, A and B may themselves be composite rules. The compiler therefore compares transition scenarios: a target scenario from A is kept only on the order region where no scenario from B occurs before it. This is why shared gates, nested guards, and absence conditions are handled in the same symbolic layer.

5 Order Regions and Likelihoods

The outcome rules above describe when a latent response becomes available. A trial likelihood asks a slightly different question: one target response is observed at time t , and all competing responses must not have occurred before that target. The exact evaluator answers this by breaking the event into cases. Each case is a region of latent finishing-time space.

The smallest pieces of a region are order facts. We call one order fact an *atom*. For example, the atoms

$$T_a < t, \quad T_a > t, \quad T_a < T_b, \quad T_a = T_b$$

say that source a finished before the observed time, after the observed time, before source b , or at the same time as source b .

A *cell* is one compatible case. It says which source releases at the observed time and which atoms must also hold. If source a releases at time t , the cell contributes the density factor $f_a(t)$. The rest of the cell then says what must be true of the other sources. For example, it may require $T_b > t$ because competitor b has not yet won, or $T_b < t$ because b must already be ready. A contradictory case, such as one requiring both $T_a < t$ and $T_a > t$, is empty and contributes nothing.

An *order region* is a union of cells. It starts from the cells for a target transition scenario, then competitor conditions refine those cells. Some

conditions simply add an atom. Others split a cell into several alternatives. After all conditions have been applied, empty cells are removed. Duplicate cells and cells already contained in another cell are removed as well. If two remaining cells would count the same event with nonzero probability, they are made non-overlapping before projection. The likelihood contribution of the region is then the sum of its cell contributions.

5.1 The simple race as a region

Suppose the observed response is A at time t , and the only competitor is B . The relevant region is

$$A \text{ releases at } t, \quad T_B > t.$$

For independent direct sources, this immediately lowers to

$$f_A(t)S_B(t).$$

With competitors $B_1, \dots, B_m$, the same reasoning gives

$$f_A(t) \prod_{j=1}^m S_{B_j}(t).$$

This familiar race likelihood is therefore not a separate idea. It is the cheapest possible order region: one density at the observed time and survival factors for the competitors.

5.2 A composite target

Now let

$$E = \text{all_of}(A, B).$$

The rule can release in two ways. Either A releases at time t and B is already ready, or B releases at time t and A is already ready:

$$\text{scenario 1: } A \text{ releases at } t, \quad T_B < t,$$

$$\text{scenario 2: } B \text{ releases at } t, \quad T_A < t.$$

If A and B are independent direct sources, these two regions project to

$$f_A(t)F_B(t) + f_B(t)F_A(t),$$

which is the density formula for `all_of()`. The important point is that the product formula is recovered from two cells. One cell says that A is the

releasing source and B is already below t ; the other says the reverse. These two cells are disjoint except for the zero-probability boundary $T_A = T_B = t$, so their contributions add. The region is the more basic object, because the same representation still works when the children are shared, nested, guarded, or blocked.

5.3 A shared-gate example

Shared structure is the place where direct product formulas become misleading. Consider

$$\text{left} = \text{all_of}(A, C), \quad \text{right} = \text{all_of}(B, C).$$

Both responses use the same latent source C . Their completion times are

$$T_{\text{left}} = \max(T_A, T_C), \quad T_{\text{right}} = \max(T_B, T_C).$$

Suppose **left** is observed at time t . It has two target scenarios:

$$\begin{aligned} A \text{ releases: } & A \text{ releases at } t, \quad T_C < t, \\ C \text{ releases: } & C \text{ releases at } t, \quad T_A < t. \end{aligned}$$

In the first scenario, C is already before t . For **right** not to be before the target, B must not have released before t , so this scenario contains the ordinary survival condition $T_B > t$.

In the second scenario, C itself is the release source. Then the competitor does not have a single independent survival factor. If $T_B > t$, **right** is later. If $T_B < t$, then B is already ready when C finishes, so **right** also releases at the same time $T_C = t$.

This creates a model tie with positive mass. It is not the event that two independent continuous variables happen to be exactly equal; that event has probability zero. The tie has mass because both composite outcomes inherit the same release time from the same latent source. For a fixed observed time t , the cell

$$C \text{ releases at } t, \quad T_A < t, \quad T_B < t$$

has density proportional to $f_C(t)F_A(t)F_B(t)$, so it contributes real density to the likelihood. The compiler therefore keeps this case as an explicit cell instead of multiplying the target by a precomputed competitor survival probability.

This is the main reason for order regions. They keep the joint latent state visible until the target and competitor relation has been expressed correctly. Only after that does the evaluator simplify and project the region.

5.4 Cells and projection

A cell C contributes one numerical term to the density at the observed time t . We write that term as $I_C(t)$. It is not a new model component; it is just the amount of likelihood density contributed by that one case.

Some cells project immediately. For example, the simple race cell

$$A \text{ releases at } t, \quad T_B > t$$

has contribution

$$I_C(t) = f_A(t)S_B(t).$$

There is no integral because the only unobserved fact, $T_B > t$, collapses to a survival function.

Other cells leave latent times that must still be integrated over. A compact way to write such a contribution is

$$I_C(t) = \int_{u \in C(t)} \prod_{a \in D_C} f_a(u_a) \prod_{q \in Q_C} h_q(u, t) du,$$

where:

- u is the vector of latent finishing times that remain after simple projections have been done.
- $u \in C(t)$ means that those latent times must satisfy all atoms in the cell, with the observed time held fixed at t .
- D_C indexes the latent sources whose densities remain inside the integral.
- Q_C indexes the other factors attached to the cell.
- Each $h_q(u, t)$ is one of those other factors. Depending on the cell, it may be a CDF, survival function, interval probability, observed density factor, or gate.

The planner removes variables from this expression symbolically whenever it can. For example,

$$T_a < t \mapsto F_a(t), \quad T_a > t \mapsto S_a(t), \quad s < T_a < t \mapsto F_a(t) - F_a(s).$$

Thus a simple cell becomes cheap math. Only a genuinely coupled cell remains as an integral.

For continuous independent leaves, an equality such as $T_a = T_b$ has zero probability and can usually be discarded as a boundary. But an equality caused by shared latent identity is different: it is the same random time appearing in two rules. The framework therefore distinguishes ordinary zero-measure equalities from positive-mass model ties.

5.5 From regions to a trial likelihood

Suppose the model defines outcomes $R_1, \dots, R_J$. If response r is observed at time t , the likelihood sums over all target scenarios that can make outcome r available at that time:

$$\ell_r(t) = \sum_{m \in M_r} I_{r,m}^{\text{win}}(t). \quad (7)$$

Here M_r is the set of target scenarios for outcome r , and $I_{r,m}^{\text{win}}(t)$ is the region contribution for scenario m occurring at t while all competing outcomes are not before it.

The familiar independent race likelihood is the special case

$$\ell_r(t) = f_{R_r}(t) \prod_{j \neq r} S_{R_j}(t). \quad (8)$$

The general formula is still the same idea, but the competitor non-win event is represented jointly with the target event before any CDF or survival factor is formed.

No-response mass. Some models leave positive probability on “no observed response”, for example because a trigger fails or because observed labels are remapped away. Before any observation remapping, the finite response probabilities satisfy

$$p_{\text{none}} = 1 - \sum_{r=1}^J \int_0^\infty \ell_r(t) dt. \quad (9)$$

So the no-response mass is whatever probability remains after summing the finite latent response probabilities.

6 Onsets

Onset parameters control when a leaf is allowed to begin. They do not change the post-start distribution itself. They only change how that post-start distribution is placed in time.

Absolute onset. A leaf may have an absolute onset delay $d_a \geq 0$. If T_a^* is the baseline finishing time after the process has started, then

$$T_a = d_a + T_a^*.$$

Equivalently,

$$F_a(t) = F_a^*(t - d_a), \quad f_a(t) = f_a^*(t - d_a),$$

with both terms equal to 0 for $t \leq d_a$. So an absolute onset simply shifts the finishing-time distribution to the right.

Chained onset. A leaf may also start only after another source has finished. If leaf a starts after source U , with an additional lag $\delta \geq 0$, then

$$T_a = T_U + \delta + T_a^*,$$

where T_a^* is the leaf's own post-start finishing time. When U has density f_U , this gives the convolution

$$f_a(t) = \int_0^{t-\delta} f_U(s) f_a^*(t - \delta - s) ds, \quad (10)$$

and similarly

$$F_a(t) = \int_0^\infty F_a^*(t - \delta - s) f_U(s) ds. \quad (11)$$

In the package, the source U may be an accumulator or pool. Once the delayed leaf has been defined, it participates in the same order-region machinery as any other leaf.

7 Triggers

Triggers control whether a source starts at all. Independent trigger draws factor across members. Shared trigger draws introduce a trial-level on/off state across a group of leaves.

Independent trigger. If leaf a has start probability $1 - q_a$, then the finite-time distribution is scaled by that probability:

$$F_a(t) = (1 - q_a)F_a^\dagger(t), \quad f_a(t) = (1 - q_a)f_a^\dagger(t). \quad (12)$$

The survival function includes both unfinished started trials and failed-start trials:

$$S_a(t) = q_a + (1 - q_a)S_a^\dagger(t).$$

Shared trigger. If several leaves share one trigger with failure probability q_g , then they either all start together or all fail together. The trial likelihood is a mixture over trigger states:

$$L = (1 - q_g)L^{(\text{on})} + q_gL^{(\text{off})}. \quad (13)$$

With several shared triggers, the exact evaluator enumerates the possible trigger states and weights their likelihood contributions. This is a source of joint dependence, because leaves controlled by the same trigger do not fail independently.

8 Observation Layer

8.1 Mixtures

A component c defines one variant of the model, with weight π_c . Mixtures can enter the likelihood in two different ways.

Explicit mixture. If the component is observed on a trial, then the trial contribution is simply

$$L = L_c.$$

This is just a model with known trial types. Each trial is assigned to one component, and the likelihood conditions on that assignment.

Marginalized mixture. If the component is not observed, then each trial is treated as arising from one of several latent process variants, and the likelihood averages over them:

$$L = \sum_c \pi_c L_c, \quad \sum_c \pi_c = 1. \quad (14)$$

Weights may be fixed or supplied through trial-wise parameters. The run-time context stores only the compact mixture plan needed for evaluation.

8.2 Outcome remapping and guessing

The outcome generated by the model need not be the same as the label that is observed. Let M_{yr} be the probability that latent outcome r is recorded as observed label y . When the response time is kept,

$$\ell_y(t) = \sum_r M_{yr} \ell_r(t). \quad (15)$$

So the observed density is a weighted sum of latent outcome densities.

If the observed label is kept but the response time is discarded, then the contribution becomes the integrated mass

$$p_y = \int_0^\infty \ell_y(t) dt. \quad (16)$$

If no response label and no response time are observed, the observation layer uses the corresponding no-response mass.

8.3 Ranked responses

The package can also use data in which the first K responses are observed on each trial:

$$(r_1, t_1), \dots, (r_K, t_K), \quad 0 < t_1 < \dots < t_K.$$

The ranked likelihood is evaluated sequentially. After observing response r_1 at time t_1 , the internal sequence state is advanced; then the next step evaluates the probability of response r_2 at time t_2 given that state, and so on. In shorthand,

$$L_{\text{rank}} = \prod_{m=1}^K \ell(r_m, t_m \mid H_{m-1}), \quad (17)$$

where H_{m-1} is the history and latent sequence state after the first $m-1$ observed responses.

For direct, disjoint outcomes this reduces to the familiar simple formula

$$L_{\text{rank}} = \left[\prod_{m=1}^K f_{r_m}(t_m) \right] \left[\prod_{j \notin \{r_1, \dots, r_K\}} S_j(t_K) \right]. \quad (18)$$

The exact implementation uses the more general sequential step distribution, so structured outcomes can still be represented through the same compiled transition machinery.

9 Compiled Evaluation

The exact evaluator follows one path:

model formula $\rightarrow$ symbolic transition scenarios
 $\rightarrow$ winner/non-winner order regions
 $\rightarrow$ canonical region cells
 $\rightarrow$ source projection
 $\rightarrow$ compiled math.

This last object is a numerical program. It contains constants, sums, products, leaf densities, CDFs, survival functions, expression distributions, and integrals where needed. The planning work is done when the likelihood context is built. Repeated likelihood calls then reuse the same compiled structure and mostly pay the mathematical evaluation cost.

Prepared data may also compress repeated trials. In that case, a compact trial with weight w_i contributes

$$w_i \log L_i$$

to the total log-likelihood. If an `ok` vector is supplied, it is interpreted on the compact trials: an excluded compact trial contributes w_i times the minimum log-likelihood value.